

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Write a Program Using JavaScript (Arrays, Functions)

2. AIM

To write and execute JavaScript programs that demonstrate the use of arrays and functions, including common array operations and different ways of defining and calling functions.

3. OBJECTIVE

- To understand how arrays are declared and used to store multiple values in JavaScript.
 - To perform common array operations such as adding, removing, searching, sorting, and reversing elements.
 - To iterate over array elements using a loop and array methods such as `forEach()`.
 - To understand how to define and call functions using different parameter styles (no parameter, single, multiple, default).
 - To use function expressions and arrow functions as alternative ways of writing functions.
 - To apply higher-order array functions — `map()`, `filter()`, and `reduce()` — to process array data.
 - To observe how `console.log()` is used to display program output at each step.
-

4. THEORY

Array

An array in JavaScript is a single variable that can store multiple values, called elements, in an ordered list. Elements are accessed using an index that starts from 0. Arrays come with built-in methods such as `push()`, `pop()`, `unshift()`, and `shift()` to add or remove elements from either end, and `sort()`, `reverse()`, and `join()` to rearrange or combine elements. JavaScript also provides higher-order array functions like `forEach()`, `map()`, `filter()`, and `reduce()` that take another function as an argument to process array data — `forEach()` runs code for every element, `map()` creates a new array by transforming elements, `filter()` creates a new array of elements meeting a condition, and `reduce()` combines all elements into a single accumulated value.

Function

A function is a reusable block of code that performs a specific task. Functions can be declared with no parameters, a single parameter, multiple parameters, or a default parameter that is used

when no argument is supplied. A function can optionally return a value using the return keyword, which can then be used elsewhere in the program. Besides regular function declarations, JavaScript also allows a function expression, where a function is assigned to a variable, and an arrow function, which offers a shorter syntax using the => symbol. Both are commonly used for short, simple operations.

5. ALGORITHM / PROCEDURE

1. Create a new JavaScript file for the array program.
 2. Declare an array using square brackets and print the original array using console.log().
 3. Access array elements using their index, including the first and last elements.
 4. Add elements to the array using push() (at the end) and unshift() (at the beginning).
 5. Remove elements from the array using pop() (from the end) and shift() (from the beginning).
 6. Find the number of elements using the length property.
 7. Search for an element using indexOf() and check its presence using includes().
 8. Display all elements using a for loop.
 9. Sort the array using sort() and reverse it using reverse().
 10. Convert the array into a single string using join() and print the result.
 11. Create a separate JavaScript file for the functions program.
 12. Define functions using different styles: no parameters, one parameter, multiple parameters, and a return value.
 13. Define a function with a default parameter value and call it with and without an argument.
 14. Write the same logic using a function expression and an arrow function.
 15. Apply forEach() to print each element of an array.
 16. Apply map() to transform array values, filter() to select values based on a condition, reduce() to accumulate a total.
 17. Run both files using Node.js (or a browser console) and verify the output for each statement.
 18. Record the observed output for a few key operations as test cases.
-

6. PROGRAM

Program 1: Array Operations (arrays.js)

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];
console.log("Original Array:", fruits);

// Access
console.log("First Fruit:", fruits[0]);
console.log("Last Fruit:", fruits[fruits.length - 1]);

// Add
fruits.push("Pineapple");
console.log("After Push:", fruits);

// Remove
fruits.pop();
console.log("After Pop:", fruits);
```

```
// Add at beginning
fruits.unshift("Watermelon");
console.log("After Unshift:", fruits);

// Remove from beginning
fruits.shift();
console.log("After Shift:", fruits);

// Length
console.log("Number of Fruits:", fruits.length);

// Search
console.log("Index of Mango:", fruits.indexOf("Mango"));
console.log("Is Apple Available:", fruits.includes("Apple"));

// Display using loop
console.log("All Fruits:");
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}

// Sort
fruits.sort();
console.log("Sorted Fruits:", fruits);

// Reverse
fruits.reverse();
console.log("Reverse Fruits:", fruits);

// Join
console.log("Fruit List:", fruits.join(", "));
```

Program 2: Functions (functions.js)

```
// 1. Function with no parameter
function welcome() {
    console.log("Welcome to JavaScript");
}
welcome();

// 2. Function with one parameter
function greet(name) {
    console.log("Hello " + name);
}
greet("Akshaya");

// 3. Function with multiple parameters
function add(a, b) {
    console.log("Sum:", a + b);
}
```

```
add(10, 20);

// 4. Function with return value
function square(n) {
  return n * n;
}
console.log("Square:", square(5));

// 5. Function with string parameter
function student(name, branch) {
  console.log("Name:", name);
  console.log("Branch:", branch);
}
student("Akshaya", "AIML");

// 6. Function with default parameter
function message(name = "Student") {
  console.log("Hello " + name);
}
message();
message("Akshaya");

// 7. Function Expression
let multiply = function(a, b) {
  return a * b;
};
console.log("Multiplication:", multiply(5, 4));

// 8. Arrow Function
let divide = (a, b) => {
  return a / b;
};
console.log("Division:", divide(20, 5));

// 9. forEach() function
let marks = [80, 75, 90, 85];
marks.forEach(function(mark) {
  console.log("Mark:", mark);
});

// 10. map() function
let numbers = [1, 2, 3, 4, 5];
let double = numbers.map(function(n) {
  return n * 2;
});
console.log("Doubled:", double);

// 11. filter() function
let ages = [12, 18, 15, 21, 25];
let adults = ages.filter(function(age) {
```

```

    return age >= 18;
  });
  console.log("Adults:", adults);

// 12. reduce() function
let values = [10, 20, 30, 40];
let total = values.reduce(function(sum, value) {
  return sum + value;
}, 0);
console.log("Total:", total);

```

7. CODE EXPLANATION

Array Code Explanation

Method / Property	Purpose
push()	Adds an element to the end of an array.
pop()	Removes the last element from an array.
unshift()	Adds an element to the beginning of an array.
shift()	Removes the first element from an array.
length	Property that returns the total number of elements in an array.
indexOf()	Returns the index position of the first matching element, or -1 if not found.
includes()	Returns true or false depending on whether a value exists in the array.
sort()	Sorts array elements alphabetically or numerically.
reverse()	Reverses the order of elements in an array.
join()	Combines all array elements into a single string, separated by the given separator.
forEach()	Runs a given function once for every element in an array, without returning a new array.
map()	Creates a new array by applying a function to every element of the original array.
filter()	Creates a new array containing only the elements that satisfy a given condition.
reduce()	Reduces all array elements to a single accumulated value, such as a sum or total.

Function Code Explanation

Concept	Purpose
function name() {...}	Declares a function with no parameters.
function greet(name) {...}	Function with a single parameter.
function add(a, b) {...}	Function with multiple parameters.
return n * n;	Returns a value from the function using the return keyword.

<code>name = "Student"</code>	Default parameter value used when no argument is passed during the function call.
<code>let multiply = function(a, b) {...}</code>	Function expression — stores an anonymous function inside a variable, called using the variable name.
<code>(a, b) => { return a / b; }</code>	Arrow function — a shorter syntax for writing functions, commonly used for simple operations.

8. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Run the array program	Original array and all update operations (push, pop, unshift, shift) print correctly	Console displayed the original array followed by the correctly updated array after each operation	Pass
TC-02	Check <code>indexOf()</code> and <code>includes()</code> output	Correct index of "Mango" and true/false for "Apple"	Printed index of Mango correctly and true for Apple's presence	Pass
TC-03	Run <code>sort()</code> , <code>reverse()</code> , and <code>join()</code>	Array is sorted alphabetically, then reversed, then joined into a single string	Sorted array, reversed array, and comma-separated string all printed as expected	Pass
TC-04	Call <code>message()</code> with and without an argument	Uses default value "Student" when no argument is passed, otherwise uses the given name	Printed "Hello Student" and "Hello Akshaya" correctly	Pass
TC-05	Run <code>map()</code> , <code>filter()</code> , and <code>reduce()</code> on sample arrays	Doubled array, filtered adults list, and total sum print correctly	Console showed the doubled numbers, the filtered adult ages, and the correct total sum	Pass

9. OUTPUT

Output 1: Array Operations Output

```

PS C:\Users\syama\OneDrive\Desktop\122> node .\exp3_1.js
Last Fruit: Grapes
After Push: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes', 'Pineapple' ]
After Pop: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
After Unshift: [ 'Watermelon', 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
After Shift: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
Number of Fruits: 5
Index of Mango: 2
Is Apple Available: true
All Fruits:
Apple
Banana
Mango
Orange
Grapes
Sorted Fruits: [ 'Apple', 'Banana', 'Grapes', 'Mango', 'Orange' ]
Reverse Fruits: [ 'Orange', 'Mango', 'Grapes', 'Banana', 'Apple' ]
Fruit List: Orange, Mango, Grapes, Banana, Apple
PS C:\Users\syama\OneDrive\Desktop\122> 

```

Output 2: Functions Output

```

PS C:\Users\syama\OneDrive\Desktop\122> node .\exp3_2.js
Welcome to JavaScript
Hello Akshaya
Sum: 30
Square: 25
Name: Akshaya
Branch: AIML
Hello Student
Hello Akshaya
Multiplication: 20
Division: 4
Mark: 80
Mark: 75
Mark: 90
Mark: 85
Doubled: [ 2, 4, 6, 8, 10 ]
Adults: [ 18, 21, 25 ]
Total: 100
PS C:\Users\syama\OneDrive\Desktop\122> 

```

10. RESULT

Thus, JavaScript programs demonstrating arrays and functions were successfully written and executed. Various array operations such as adding, removing, searching, sorting, and reversing elements were performed, and functions were defined using different parameter styles, function expressions, arrow functions, and higher-order array methods (forEach, map, filter, reduce). The output of each program was verified and found to be correct.